

Enhancing Autonomous Intrusion Detection System with Generative Adversarial Networks

Kevin Kostage ^{*}, David West ^{*}, Tim Meinert ^{*}, Chengyi Qu ^{*†}, Prasad Calyam [‡], Luca Mazzola [§]

^{*†} Department of Computing and Software Engineering, Florida Gulf Coast University - Florida, USA.

[‡] Department of Electrical Engineering and Computer Science, University of Missouri - Columbia, USA.

[§] Lucerne University of Applied Sciences and Arts - Lucerne, Switzerland

Email: ^{*}{kstkostage9457, dawest6674, trmeinert9424}@eagle.fgcu.edu, [†]cqu@fgcu.edu,

[‡]calyamp@missouri.edu, [§]luca.mazzola@hslu.ch

Abstract—Effective training in Machine Learning and Deep Learning models necessitates datasets that provide sufficient patterns and contextual information, particularly crucial in IoT networks. Imbalanced datasets, however, significantly challenge the performance of Autonomous Intrusion Detection Systems (IDS), leading to suboptimal detection rates for minority classes. In this paper, we address this issue by utilizing various Generative Adversarial Network (GAN) models, including WGANP, CGAN, CTGAN, and CWGANP, to generate synthetic data that balance these imbalanced datasets. We evaluate the performance of IDS models trained on GAN-augmented datasets against those trained on unbalanced datasets, considering metrics such as fitting duration, generation duration, accuracy, precision, recall, and F1 score. Our findings reveal substantial improvements in IDS performance with the application of GANs across binary, general, and specific attack classifications. Additionally, we compare the effectiveness of GANs with classical sampling algorithms, such as SMOTE and Random Oversampling. This comprehensive evaluation underscores the potential of GANs as a sophisticated solution for improving IDS accuracy and reliability in handling complex and highly imbalanced datasets.

Index Terms—Intrusion Detection Systems, Generative Adversarial Networks, Cybersecurity Datasets, Classification

I. INTRODUCTION

In our increasingly interconnected world, cybersecurity attacks are becoming more frequent and costly. In 2023 alone, there were 3,205 data breaches, compromising the data of 353,027,392 victims, a 77% increase over the previous year [1]. This surge in cybersecurity attacks has led to an average cost of \$4.45 million per breach in the U.S., underscoring the critical need for effective cybersecurity measures [2]. As our reliance on IoT devices grows, the attack surface for cyber threats expands, introducing additional vulnerabilities. IoT devices, while convenient, create numerous entry points for attackers, complicating the task of securing networks [3].

Intrusion Detection Systems (IDS) are vital for protecting networks, monitoring traffic in real-time to identify suspicious activities. Traditional IDS models use a rules-based approach to detect known attack patterns or anomalies in normal traffic behavior [4]. However, the limitations of these models have led to the adoption of Autonomous IDS models, which utilize algorithms to improve detection capabilities. Autonomous IDS models can detect novel attacks that lack known signatures and reduce false positives, enhancing the security of IoT

networks [5], [6]. However, the performance of Autonomous IDS models is heavily dependent on the quality and balance of their training datasets.

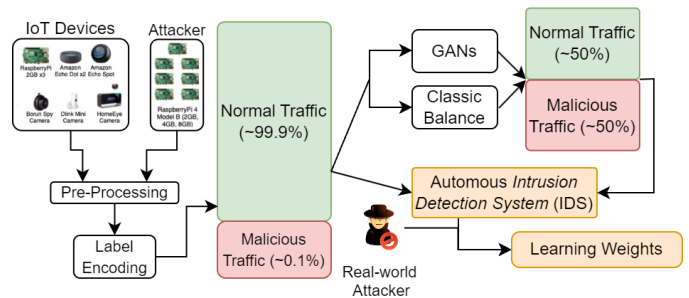


Figure 1: Overview of Data Balancing and IDS Enhancement Process Using GANs and Classic Balance Techniques.

Imbalanced datasets, where normal traffic vastly outnumbers malicious traffic, pose a significant challenge to Autonomous IDS models. This imbalance can cause models to overlook minority classes, leading to poor detection rates for rare but critical attack types [7]. To address this, we propose the use of Generative Adversarial Networks (GANs) to generate synthetic data and balance the datasets. Figure 1 illustrates the data processing and model training workflow for enhancing an Autonomous IDS using GAN. The process begins with data collection from various IoT devices, including e.g., Raspberry Pi cameras, Amazon Echo devices, and other smart cameras. This data, predominantly normal traffic ($\approx 99.9\%$), undergoes pre-processing and label encoding to differentiate between normal and malicious traffic ($\approx 0.1\%$). To address the imbalance, the data is fed into two parallel processes: classical balancing techniques and GAN-based data augmentation. These methods generate balanced datasets with approximately equal proportions of normal and malicious traffic ($\approx 50\%$ each). The balanced datasets are then used to train the IDS, which adjusts its learning weights based on the data.

In this paper, we propose the use of various GAN models to generate synthetic data and balance the cybersecurity attack datasets, thereby enhancing the ability of the Autonomous IDS to detect and respond to various cyber-attacks. By employing various GAN models, we aim to enhance the representation

of malicious traffic in training datasets. Our approach compares the effectiveness of GAN-augmented datasets against traditional data balancing techniques like SMOTE and Random Oversampling, demonstrating the potential of GANs to significantly improve IDS performance in detecting a wide range of cybersecurity attacks. Our findings reveal substantial improvements in IDS performance through the application of GANs across binary, general, and specific attack classifications. Furthermore, a comparative analysis with classical sampling algorithms, such as SMOTE and Random Oversampling, highlights the superior efficacy of GANs in balancing datasets. This comprehensive evaluation underscores the potential of GANs as an advanced and effective solution for enhancing IDS accuracy and reliability, particularly in the context of complex and highly imbalanced datasets.

II. RELATED WORK

Cybersecurity attack data collection and database analytics have been investigated for a long period of time [8]. However, the use of IoT and edge devices has diminished existing collected datasets and findings due to the significant evolution in network attack techniques [9]. In our research, we utilize cutting-edge cybersecurity attack datasets such as CICIOT2023 [10] to provide more relevant and effective solutions for today's security landscape. The inherent imbalance in the CICIOT2023 dataset mirrors the common issue in cyberattack datasets, where the prevalence of certain attacks over others leads to skewed data [11]. IoT devices, in particular, experience specific types of attacks more frequently, exacerbating this imbalance. Such skewed datasets can impair the performance of IDS models, which struggle to detect less common attack types. Our work addresses this by using advanced data augmentation techniques to ensure a more balanced and representative dataset, thereby enhancing the detection capabilities of IDS models.

Intrusion Detection Systems (IDS) are critical components of network security, designed to monitor network traffic in real-time and identify potential threats [5]. Autonomous IDS leverages machine learning algorithms to analyze network traffic data and improve detection rates [12]. In our paper, we considered an integrated autonomous IDS that included various groups of ML-based methods to measure the performance of the classical sampling and GAN model approaches to balancing the training data. The times of fitting and generation of data for each classical sampler and GAN model to compare the efficiency of the approach with the IDS.

Classical sampling techniques are commonly utilized for balancing data in machine-learning models. Oversampling methods, such as random oversampling and SMOTE [6], increase the number of instances in the minority class by duplicating or slightly altering existing data. However, these techniques can result in unrealistic datasets, potentially affecting model accuracy for underrepresented classes. Under-sampling methods, like random undersampling and cluster centroids [13], reduce the majority class to match the size of

the minority class, which can lead to a loss of critical information and negatively impact IDS performance. Additionally, GANs offer a sophisticated alternative by generating new data based on learned patterns from the training data. Despite their potential, GANs are infrequently used in cybersecurity due to their complexity and computational demands. Furthermore, there are variants of the GAN model that trains the model differently based on components and modifications added to the model. Each GAN variant has a different approach to handling the tabular training data during training which affects the balance and realism of the generated data and performance of the model. Conditional GANs (CGAN) [14] and Conditional Wasserstein GANs with Gradient Penalty (CWGANP) [15] can generate data for specified classes, ensuring balanced datasets for IDS training [16]. CTGAN [17] and Wasserstein GANs with Gradient Penalty (WGANP) [18] focus on creating realistic data but can achieve balanced datasets through batch generation and undersampling. This makes GANs a promising tool for improving IDS models' performance on imbalanced datasets.

III. DATASETS AND FRAMEWORK OVERVIEW

A. Cybersecurity Attack Databases and Processing Methods

In our database collection phase, we investigated three IoT cybersecurity datasets for analyzing and generating synthetic cybersecurity attack data, i.e., CIC EV Charger Attack 2024 (CICEVSE2024) [19], CIC IoV 2024 [20], and CIC IoT 2023 (CICIOT-2023) [10] datasets. Each dataset is processed and analyzed following a consistent methodology to ensure robust and reliable findings. In this section, we detailed the CICIOT-2023 dataset, with similar analyses conducted for the other two datasets.

The CICIOT-2023 dataset comprises 33 attack labels categorized into 7 general attack classes and a single benign class. This dataset includes 47 attributes, of which 46 are features and one is a label. The features are further divided into 22 numerical and 24 categorical attributes. An analysis of the label distribution, as shown in Table I, reveals that DDoS attacks are predominant, whereas brute force attacks represent the smallest minority class. The cybersecurity attack data were collected from 105 IoT devices equipped with various sensors, including cameras, microphones, and motion sensors.

The CICIOT-2023 dataset is utilized for training Generative Adversarial Networks (GANs) to generate balanced synthetic cybersecurity attack datasets. The dataset is preprocessed by the developers prior to download, involving steps such as feature extraction, data cleaning, combining DPKT files into CSV files, assigning traffic labels, and creating batch sizes for the data.

A specified subset of the 165 CSV files in the CICIOT-2023 dataset is randomly sampled and added to a data frame. This subset is then triplicated: one copy retains the original 33 attack-type labels plus benign traffic, the second copy maps the labels to 7 attack categories (e.g., DDoS, Mirai) plus benign, and the third copy maps the labels to a binary classification (attack vs. benign).

Table I: Breakdown of Example IoT Attack Dataset by Attack Category and Imbalance Ratio Analysis

Attack Category	Total Rows	Imbalance Ratio	No. of Attack Types	Most Common Attack Types	Total Rows
DDoS	33,984,560	1:1	12	UDP Flood	5,412,287
DoS	8,090,738	4.2:1	4	TCP Flood	2,671,445
Recon	354,568	95.84:1	5	Host Discovery	134,378
Web-Based	24,829	1,368:1	5	Browser Hijacking	5859
Brute Force	13064	2,601:1	-	-	-
Spoofing	486,504	70:1	2	ARP Spoofing	307,593
Mirai	2,634,124	127:1	3	Greeth Flood	991,866

The dataset's numerical features are normalized using a MinMax scaler, which is saved for future analysis or visualization of synthetic or sampled datasets. The labels are encoded, and the mapping is preserved for later decoding. Before training, the GAN model verifies the labels. After generating synthetic data based on a specified sample size, the features are unscaled and the labels are decoded.

B. Framework Overview

Figure 2 depicts a framework that addresses the challenge of imbalanced datasets in training models for classification or anomaly detection, particularly in the context of IoT devices. The framework leverages a Generative Adversarial Network (GAN) to generate synthetic data to balance the dataset, thereby improving the accuracy and robustness of the classification models.

When training a model to classify or detect anomalies, the distribution of labels is crucial. An imbalanced dataset, where some classes are underrepresented, leads to a model biased towards the majority class. This results in poor performance in detecting or classifying minority classes. For IoT devices, generating enough real-world attack traffic to balance the dataset is challenging and time-consuming. Thus, using GANs to generate synthetic data offers an efficient solution.

In the training dataset pre-processing stage, the initial dataset contains real-world data. A subset of this dataset is randomly sampled to create a manageable training set. The selected classification type focuses on specific types of classifications, such as attack types. The numerical features of the dataset are normalized to ensure uniformity and improve model training. The dataset is then split into features and labels, with the labels encoded to numerical values suitable for the model.

The GAN model comprises two main components: the generator and the discriminator. The process begins with a noise vector, which serves as the random input for the generator. The generator produces synthetic data, specifically attack data, from this noise vector. The discriminator's role is to evaluate the generated data against the real training dataset to distinguish between real and synthetic data. The discriminator determines whether the data is real or generated, and this feedback loop helps refine the generator's output. The generated attack data is then used to augment the training dataset. We detailed our GAN models with the comparison of several classical sampling algorithms in Section IV.

In the comparative analysis stage, classifiers are trained on both the synthetic training dataset (augmented with synthetic

instances) and the real-world training dataset. The performance of these classifiers is compared using specific metrics to evaluate their effectiveness. This stage helps determine how well the synthetic data aids in balancing the dataset and improving model performance. Similarly, the autonomous comparative analysis involves a classifier model used for the final analysis. Performance metrics derived from the classifier are used to autonomously compare classifiers, ultimately determining the best-performing model. This approach ensures a balanced dataset through sophisticated oversampling, improving the model's ability to accurately classify or detect anomalies across all classes. Detailed comparative analysis algorithm design and development can be checked in Section V.

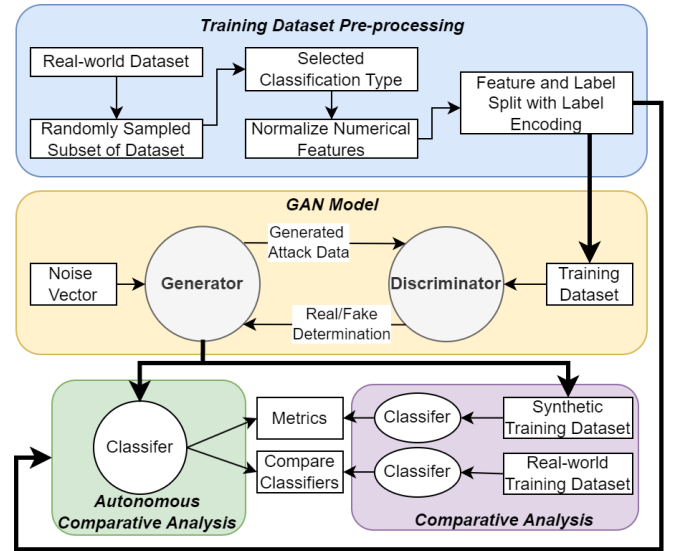


Figure 2: Framework for Balancing Imbalanced Datasets using Generative Adversarial Networks (GANs) for Enhanced Classification and Anomaly Detection in IoT Devices

IV. GAN-BASED DATASET BALANCING MODEL

In this section, we first discuss the classical sampling algorithms employed for dataset balancing before exploring the rationale for using Generative Adversarial Networks (GANs) to balance cybersecurity attack datasets. We also compare various GAN model variants through preliminary experiments to highlight their efficacy.

A. Classical Sampling Algorithms

Before we delve into the GAN model discussion, we first investigate the classical sampling algorithms commonly used

for tabular datasets. These algorithms are categorized as either oversampling methods or undersampling methods.

Oversampling is a data augmentation technique where the dataset's minority classes are enhanced by adding more samples until all classes match the representation of the largest majority class. Several methods are used to achieve this. Random Oversampling involves copying random instances of a class's data to generate additional samples. While computationally inexpensive, this method may introduce overfitting, especially if certain data clusters are oversampled disproportionately. The Synthetic Minority Oversampling Technique (SMOTE) generates synthetic data points by interpolating between a sample and its same-class neighbors. This method mitigates the risk of overfitting associated with random oversampling by creating more diverse synthetic samples, although it requires higher processing overhead. While data augmentation through techniques like oversampling and SMOTE can help balance datasets, it also introduces potential pitfalls. The generation of synthetic data points can lead to unrealistic datasets that may not accurately represent real-world scenarios due to either copying existing instances or using limited patterns. This is particularly problematic in sensitive applications such as cybersecurity, where the authenticity of data is crucial for model performance.

Undersampling balances dataset classes by reducing the number of samples from majority classes to match the smallest minority class. This approach also includes several methods. Random Undersampling involves randomly selecting and removing samples from the majority class until the class balance is achieved. The main advantage of this technique is its speed; however, it can lead to the loss of valuable information about the class distribution, potentially diminishing the dataset's representativeness. The Cluster Centroids method uses a K-Means algorithm to identify and preserve the essential data points of the majority class while reducing the dataset size. This method provides a balanced dataset that retains significant data points but is computationally intensive, making it less suitable for very large datasets. Undersampling techniques, while effective in reducing dataset size and balancing classes, come with their own set of challenges. By removing samples from the majority classes, there is a risk of losing valuable information critical for understanding the overall distribution of the data. This loss can affect the model's ability to generalize and perform well on unseen data, as the reduced dataset may not fully capture the intricacies and variances of the original data.

Classical sampling algorithms have inherent limitations when applied to IoT attack databases. Oversampling can lead to overfitting and unrealistic datasets, whereas undersampling can result in a loss of valuable information (see Section VI-D for the detailed classical sampling evaluation results). These challenges underscore the need for more sophisticated approaches, such as Generative Adversarial Networks (GANs). GANs can generate new data based on patterns learned from the training dataset, offering a more advanced method to balance the data. Unlike classical sampling methods, GANs can produce synthetic data that maintains the complexity and

variability of the original dataset, making them particularly suitable for cybersecurity applications where data authenticity is paramount. By using GANs, we can address the limitations of classical sampling methods and create balanced datasets that improve model performance and generalization.

B. General Components of GAN

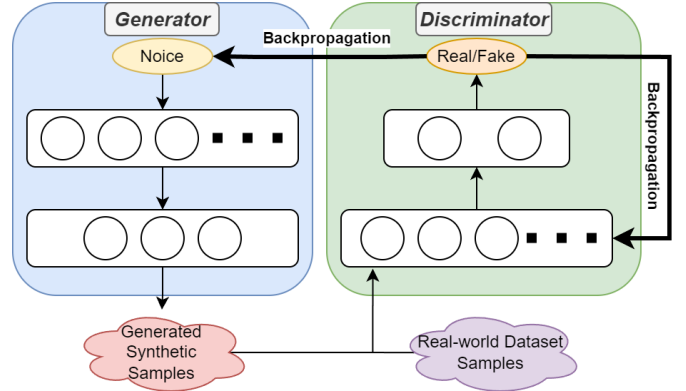


Figure 3: Generative Adversarial Network (GAN) Training Process Details for Synthetic Data Generation.

In our approach to balancing the IoT attack database, we employ a Generative Adversarial Network (GAN) model designed to create synthetic data that complements the real dataset, addressing the issue of class imbalance (see Figure 3). The GAN consists of two neural network models: the generator and the discriminator. The generator produces synthetic data, while the discriminator attempts to differentiate between real and synthetic data. During training, the generator is fed Gaussian noise, creating synthetic data that is evaluated by the discriminator. Both models use gradients to improve their performance through back-propagation, continuing until the generator produces high-quality synthetic data indistinguishable from real data.

To enhance the GAN's effectiveness, various components and variants are incorporated. The Wasserstein distance loss function is utilized to prevent mode collapse and improve training stability by considering the work required to transform the synthetic data's probability distribution into that of the real data. Conditional inputs are used to generate synthetic data that maintains the original class distribution ratios, focusing on minority classes to address class imbalance. Tabular GANs are specialized for generating tabular data, preserving the relationships and statistical properties of the training dataset. Gradient penalties are added to prevent vanishing gradients, ensuring the generator receives actionable feedback.

C. GAN Algorithm and Variants

Algorithm 1 outlines the steps for training a GAN and generating synthetic data to balance an IoT attack database. The process begins by loading a random subset of the initial dataset. If the label classes are grouped or binary, the dataset is relabeled accordingly. The dataset is then preprocessed by scaling and encoding its features. A GAN is created with specific model types and hyper-parameters, and it is

trained using the encoded dataset. For each class in the label classes, the trained GAN generates synthetic data, which is added to the synthetic dataset. This synthetic dataset is then postprocessed by reversing the scaling and encoding steps. The final synthetic dataset is exported to a file, and a comparison report between the initial and synthetic datasets is generated.

Algorithm 1 GAN Training and Synthetic Data Generation

Initialization:

Dataset $\leftarrow$ Load a Random Subset(Initial Dataset)

Procedure:

if *Label Classes are Grouped or Label Classes are Binary* **then**

 | Dataset $\leftarrow$ Relabel(Dataset, Label Classes)

end

Preprocessed Dataset $\leftarrow$ Scale and Encode(Dataset)

GAN $\leftarrow$ Create GAN(Model Type, Hyper-Parameters)

Trained GAN $\leftarrow$ Fit GAN(Encoded Dataset)

for *Class in Label Classes* **do**

 Synthetic Class Data $\leftarrow$ Use GAN to Create Synthetic Data(Trained GAN, Class)

 Synthetic Dataset $\leftarrow$ Synthetic Dataset + Synthetic Class Data

end

Postprocessed Synthetic Dataset $\leftarrow$ Inverse Scale and Inverse Encode(Synthetic Dataset)

Synthetic Dataset File $\leftarrow$ Export(Postprocessed Synthetic Dataset)

Report File $\leftarrow$ Make a Comparison Report(Initial Dataset, Synthetic Dataset)

Our designed GAN algorithms include variants to leverage their respective strengths. The Wasserstein GAN with Gradient Penalty (WGAN-GP) combines the Wasserstein loss function with a gradient penalty for improved training dynamics. The Conditional GAN (CGAN) uses conditional inputs to generate synthetic records in specified label ratios, ideal for data augmentation. The Conditional Tabular GAN (CTGAN) employs both conditional inputs and an architecture adapted for tabular data. The Conditional Wasserstein GAN with Gradient Penalty (CWGAN-GP) merges the benefits of the Wasserstein loss function, gradient penalty, and conditional inputs, enhancing training stability and conditional generation.

This sophisticated approach allows for the creation of realistic, high-quality synthetic data that preserves the complexity and variability of the original dataset, improving model performance and generalization. By employing these advanced GAN variants, we effectively address the limitations of classical sampling algorithms, ensuring accurate detection and classification of cybersecurity attacks in IoT environments.

V. AUTONOMOUS INTRUSION DETECTION MODEL

Following the training of our GAN models, the synthetic data generated is used to train various classifier models, collectively referred to as evaluators. These evaluators are tasked with predicting the labels of real data. The goal is to ensure that if the synthetic training data is indistinguishable

from real attack data, the evaluators will have similar success in predicting real data labels as those trained on purely real data. After training, each IDS model is evaluated based on its prediction performance. The model with the best performance is then selected to classify attacks in the IoT environment.

In our autonomous IDS, several methods are employed to build and evaluate the classifier models [5]. Logistic Regression is a straightforward and interpretable model. In the One vs. Rest (OvR) approach, a single classifier per class is trained, with each class against all other classes, converting a multi-class classification problem into multiple binary classification problems. This method has the hyperparameters of regularization strength (C) and the solver type. The pros include simplicity and ease of interpretation, but it can be less effective for highly imbalanced data.

Random Forest is an ensemble method that builds multiple decision trees using random subsets of features, ensuring low correlation among the trees. The key hyperparameters include the number of trees (n_estimators), the maximum depth of trees (max_depth), and the number of features to consider (max_features). The Random Forest method has robustness and high accuracy, while including potential overfitting and high computational cost.

K-Nearest Neighbor (KNN) works by finding the closest data points in the training dataset to a new data point and making predictions based on these nearest neighbors. The hyperparameters for KNN are the number of neighbors (K) and the distance metric (e.g., Euclidean distance). The pros include simplicity and adaptability to new data, while cons involve poor scalability and high memory usage.

Deep Neural Networks (DNNs) are powerful models capable of modeling complex non-linear relationships with multiple hidden layers between the input and output layers. The hyperparameters include the number of layers, the number of neurons per layer, learning rate, and batch size. The pros are their ability to learn complex patterns, but they require large amounts of data and computational resources.

The Perceptron is a linear classifier that learns by incrementally correcting mistakes made by the classifier. The base Perceptron uses hyperparameters like the learning rate and the number of iterations. The pros include simplicity and speed, but it performs poorly on non-linearly separable data. The Multilayer Perceptron extends this by using multiple layers and non-linear activation functions, with hyperparameters such as the number of layers, number of neurons, learning rate, and activation function. MLPs can model complex relationships but require more computational resources.

Adaboost is an ensemble technique that combines multiple weak learners to create a strong learner. It adjusts the weights of classifiers and training data to focus on difficult cases. The hyperparameters include the number of estimators and learning rate. The pros are improved performance on difficult datasets, but it can be sensitive to noisy data and outliers.

XGBoost is another powerful ensemble method based on gradient boosting. It builds models in stages to optimize an arbitrary differentiable loss function. The key hyperparameters

Algorithm 2 Autonomous IDS Model Evaluation

Initialization:

Synthetic Dataset $\leftarrow$ Load Dataset(Synthetic Dataset File)
Test Dataset $\leftarrow$ Load Dataset(Untrained Subset of Real Dataset)
Max F1 Score $\leftarrow$ 0
Optimal IDS $\leftarrow$ Empty Variable

Procedure:

Synthetic Dataset $\leftarrow$ Load Dataset(Synthetic Dataset File)
for IDS Model in Evaluators **do**
 IDS (Evaluator) $\leftarrow$ Create Classifier(Model)
 Trained IDS $\leftarrow$ Fit IDS(Synthetic Dataset)
 IDS Metrics $\leftarrow$ Use IDS to Predict on Labeled Data(Trained IDS, Real Data Subset)
 Metrics File $\leftarrow$ Save to File(IDS Metrics)
 Autonomous Selection:
 if Max F1 Score is empty **then**
 Max F1 Score $\leftarrow$ IDS Metrics[F1 score]
 Optimal IDS $\leftarrow$ IDS Model
 end
 if IDS Metrics[F1 score] is greater than Max F1 Score **then**
 Max F1 Score $\leftarrow$ IDS Metrics[F1 score]
 Optimal IDS $\leftarrow$ IDS Model
 end
end

include the number of trees, learning rate, maximum depth of trees, and subsample ratio. The pros include high accuracy and efficiency, while the cons are the risk of overfitting and significant computational cost.

Our Intrusion Detection System (IDS) will utilize advanced classification methods, i.e., Logistic Regression, Random Forest, K-Nearest Neighbor (KNN), Deep Neural Networks (DNN), Perceptron, AdaBoost, and XGBoost to ensure comprehensive and accurate detection of malicious traffic. By employing diverse methods, each with its unique strengths and hyperparameters, the IDS will leverage combined capabilities to analyze and classify network traffic effectively. Through an autonomous decision-making process, the IDS will evaluate the performance of each model using the GAN-generated balanced dataset and select the optimal model based on the highest F1 score. This approach ensures that the IDS can reliably distinguish between benign and malicious traffic, thereby enhancing the security and resilience of IoT environments.

Algorithm 2 outlines the process for training, evaluating, and selecting the optimal IDS model. This algorithm initializes by loading the synthetic and test datasets. It then iterates over each IDS model in the set of evaluators, training each model on the synthetic dataset. The trained IDS models are then used to predict on a labeled subset of real data, with their performance metrics saved to a file. An autonomous selection process is implemented to identify the model with the highest F1 score, which is then designated as the optimal IDS model for classifying attacks in the IoT environment. This ensures the

selection of a robust model capable of accurately detecting and classifying cybersecurity attacks based on the balanced dataset generated by the GAN.

VI. EXPERIMENT PERFORMANCE EVALUATION

In this section, we initially delve into the evaluation metrics crucial for assessing the accuracy of cybersecurity attack databases. Following this, we evaluate the efficacy of our balanced database approach, juxtaposing it against both classical dataset-balancing models and alternative GAN-based methods. Moreover, we expound on our approach's performance across various classification types, meticulously analyzing its superiority in terms of time efficiency, accuracy, precision, recall, and F1 scores across all IDS models.

A. Experiment Setup

The experimental setup utilized a local Graphics Processing Unit (GPU) and Central Processing Unit (CPU) for most trials, specifically an RTX 3080 and an Intel i7-12700k, respectively. The experiments were conducted within Linux Anaconda virtual environments. The GAN configurations were implemented using the YDataSynthetic library [21], while samplers and evaluators were created using the imbalanced-learn library [22]. Due to package conflicts between Jupyter Notebook and YDataSynthetic, separate virtual environments leveraging Windows Subsystem for Linux (WSL) were employed for the sampling techniques and GAN implementations.

Here we detail the hyperparameters selected for the classical sampling methods, GAN variants, and IDS models. The Random Oversampler was configured with a sampling strategy set to auto, which resamples all classes to match the majority class's sample size without applying shrinkage. The Random Undersampler also used a sampling strategy set to auto, resampling all classes to the minority class's sample size, with replacement set to false. For SMOTE, the sampling strategy was set to auto, with k_neighbors set to 5, n_jobs to -1 to utilize all available CPU cores, categorical features specified from the dataset, and m_neighbors set to 10 to determine if a minority sample is in danger. The ClusterCentroids configuration included a sampling strategy set to auto, the estimator set to None (defaulting to KMeans), and voting set to auto.

The GAN variants' settings were designed for consistency and to enhance balancing. Common parameters included an input noise and latent dimension set to 46 (matching the number of dataset features), with the number of columns and features also set to 46. Labels were set to 34 for specific, 8 for general, and 2 for binary. The batch size was 500, betas were 0.5 and 0.9, and the learning rate was 0.0005. The Generator submodel's trainable layers included 32 neurons in the first layer, 16 neurons in the second, and 8 neurons in the third. The Critic submodel had a single trainable layer with 32 neurons. The standard layer structure for the Generator submodel included a 46-neuron input layer, followed by layers with 32, 16, and 8 neurons, and an output layer with 46 neurons. The Critic submodel featured a 46-neuron input layer,

a 32-neuron layer, and a final single-neuron layer to determine the realism of the data.

The GAN variants had specific differences due to time constraints. The conditional settings varied, with conditions set to none, and the number of training epochs also varied: 100 epochs for CWGAN and WGAN and 25 epochs for CTGAN and CGAN. This configuration ensured a robust evaluation of the GANs' performance in generating balanced synthetic data for the IDS.

B. Experiment Workflow

The experimental workflow consists of two main processes: one for classical sampling techniques and another for training GANs to create synthetic data. These processes are designed to compare the effectiveness of classical sampling techniques with that of GAN-generated synthetic datasets. For each classification type - *binary attack vs. benign traffic (1+1)*, *general attacks vs. benign traffic (7+1)*, and *specific attacks vs. benign traffic (33+1)* - the experiments are repeated three times. This approach ensures a thorough evaluation of how each balancer handles different classifications.

In the GAN training process, the GAN architecture is built with defined input layer dimensions, layer dimensions, learning rate, and activation functions. The training data, consisting of numerical and categorical features separated from the labels, is fed into the model. The training cycle begins with the discriminator receiving input from the generator (initially noise) and comparing it with real training data to discern real from fake samples. The discriminator and generator iteratively train against each other, with the generator continually adjusting to better fool the discriminator.

Once the GAN model completes its training cycle, it generates new synthetic data. This data is conditioned on specific labels or features when required. Non-conditional models generate data based on the distribution of training data. The generated synthetic data, initially scaled and encoded, is then reverse-scaled and decoded for proper analysis. On the contrary, in the classical sampling process, real dataset samples are checked for feature presence, creating small subsets used to train classifiers. These subsets are balanced using various oversampling and undersampling techniques, with the labels encoded accordingly. The balanced datasets are split into training and testing sets for classifier evaluation.

The evaluation and analysis phase involves using the balanced synthetic and sampled datasets to train various IDS models. These models are then tested using a real dataset sample to evaluate their performance. Metrics are tracked and recorded to compare the effectiveness of the different balancing techniques. The analysis focuses on the differences in feature values produced for each label by the synthetic and sampled data.

Outliers discussion: Figure 4 illustrates the traffic label distribution for imbalanced synthetic datasets generated by two GAN variants: CTGAN and WGAN. It shows the record counts for various types of traffic, including benign, brute force, DDoS, DoS, Mirai, reconnaissance, spoofing, and web.

web traffic. Notice that CGAN and CWGAN variants, due to their conditional generation capabilities, can ensure a balanced distribution of labels in their synthetic datasets. However, the WGAN and CTGAN variants, which lack this capability, exhibit imbalanced label distributions. Despite this, the iterative generation of synthetic data batches followed by undersampling allows for the creation of a balanced dataset. This approach ensures that even without conditional generation, synthetic datasets can be adjusted to have a balanced representation of traffic types, addressing the imbalances observed in initial generations.

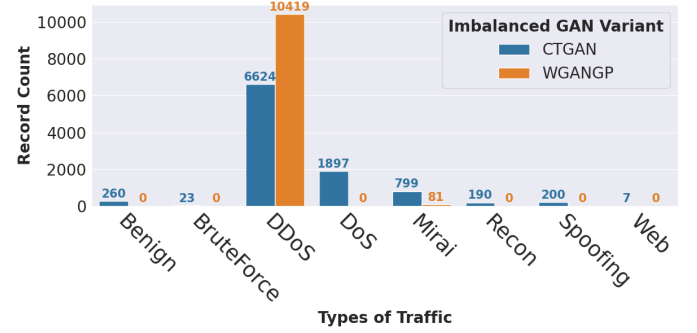


Figure 4: Label Distribution for Imbalanced Synthetic Datasets

C. Performance Metrics

In evaluating the performance of the IDS models, several metrics are employed to provide a comprehensive assessment of their effectiveness. These metrics include timing metrics and common learning algorithm metrics such as accuracy, precision, recall, and F1 score. Timing metrics are recorded for each phase of the process, including the training of models, data generation, and IDS predictions. These metrics provide insights into the computational efficiency and scalability of the models [23]. The ability of IDS models to accurately label the testing data subset is evaluated using four key metrics. These metrics, applied in their macro-versions, offer a holistic evaluation of the model's performance across all classes.

The ability of IDS models to accurately label the testing data subset is evaluated using four key metrics. These metrics, applied in their macro-versions, offer a holistic evaluation of the model's performance across all classes.

a) *Accuracy*: Measures the proportion of correctly labeled records against the total records, indicating how frequently the model was correct in its predictions. It is calculated as:

$$\text{Macro Accuracy} = \frac{1}{L} \sum_{i=1}^L \frac{TP_i + TN_i}{TP_i + FP_i + TN_i + FN_i}$$

b) *Precision*: Measures the proportion of true positives against positively labeled predictions, representing the likelihood of the model to correctly label a class. It is calculated as:

$$\text{Macro Precision} = \frac{1}{L} \sum_{i=1}^L \frac{TP_i}{TP_i + FP_i}$$

c) *Recall*: Measures the proportion of true positives against the total number of actual positive records, indicating how many instances of a class were correctly identified. It is calculated as:

$$\text{Macro Recall} = \frac{1}{L} \sum_{i=1}^L \frac{TP_i}{TP_i + FN_i}$$

d) *F1 Score*: The F1 score combines precision and recall into a single metric, providing a balanced measure that is particularly useful for evaluating models trained on imbalanced or augmented datasets. It is calculated as:

$$\text{Macro F1 Score} = \frac{1}{L} \sum_{i=1}^L \left(2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \right)$$

In the above equations, L is the total number of labels and i is a particular label. TP_i is the number of records where i is both the actual and predicted label. FP_i is the number of records where the label is predicted but matches the actual label. TN_i is the number of records where the label is neither predicted, nor the actual label. FN_i is the number of records where a particular label is incorrectly predicted.

D. Preliminary IDS Results on Classical Sampling Models

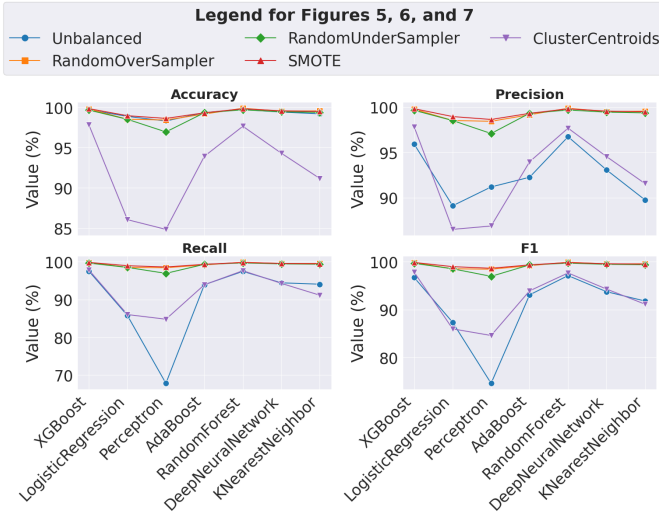


Figure 5: IDS Performance on Binary Class (binary attack vs. benign traffic (1+1)) for *Sampled* Balanced Datasets

Figures 5, 6, and 7 present the evaluation results for the unbalanced model and various classical sampling algorithms across three different datasets: specific attacks vs. benign traffic, general attacks vs. benign traffic, and binary attack vs. benign traffic. First, we can observe in Figure 5, that the unbalanced dataset yields lower performance across all metrics—accuracy, precision, recall, and F1 score—compared to the balanced datasets generated by classical sampling algorithms. Notably, SMOTE and RandomOverSampler significantly improve the performance metrics, especially precision and recall, indicating a more effective balance of the dataset.

Figure 6, assessing general attacks, shows a similar trend. The unbalanced dataset again underperforms the balanced

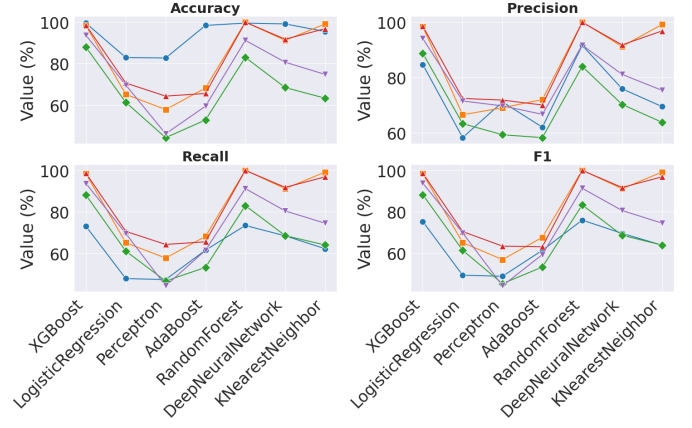


Figure 6: IDS Performance on General Class (general attacks vs. benign traffic (7+1)) for *Sampled* Balanced Datasets

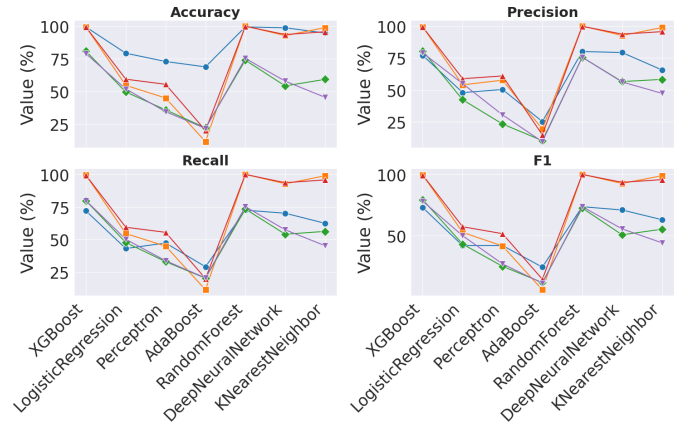


Figure 7: IDS Performance on Specific Class (specific attacks (33+1)) for *Sampled* Balanced Datasets

datasets. SMOTE and RandomOverSampler once more exhibit substantial improvements in all metrics, with ClusterCentroids and RandomUnderSampler also contributing positively but to a lesser extent. This demonstrates the consistent effectiveness of oversampling techniques in handling imbalances for this classification type.

Additionally, Figure 7, which examines specific attacks, further reinforces these findings. The unbalanced dataset's metrics are markedly lower, particularly in recall and F1 score. Here, SMOTE and RandomOverSampler provide the highest performance improvements, ensuring better accuracy and precision as well. The results for RandomUnderSampler and ClusterCentroids are slightly less impactful but still beneficial compared to the unbalanced approach.

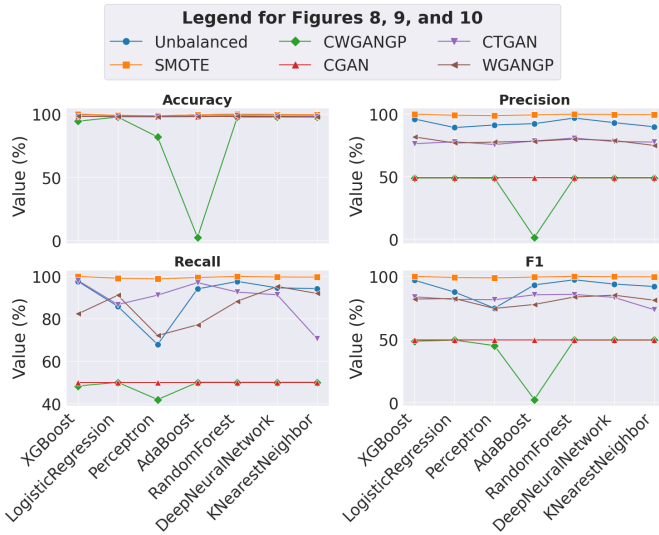
In conclusion, combining the observation from the IDS metrics and the fitting duration on the classical sampler (as shown in Table II), balancing the dataset with classical sampling algorithms markedly enhances the performance of IDS models across all evaluated metrics. The unbalanced dataset consistently underperforms, particularly in recall and F1 scores, highlighting the necessity of employing balancing techniques. Among the methods, SMOTE and Random

Table II: Fitting and Generation Duration for Various Balancing Models (*left: Classical Sampler, right: GAN Models*)

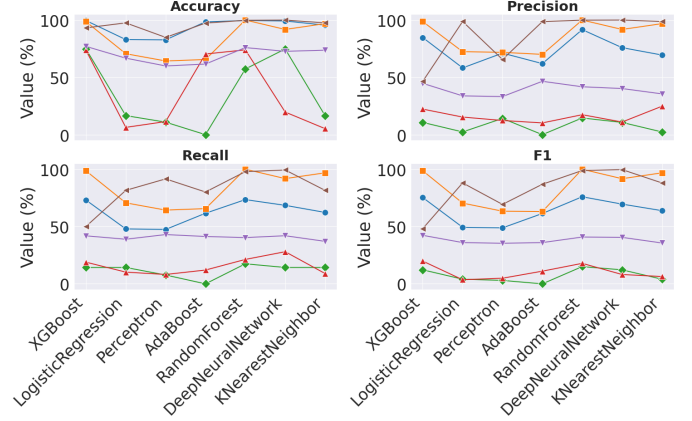
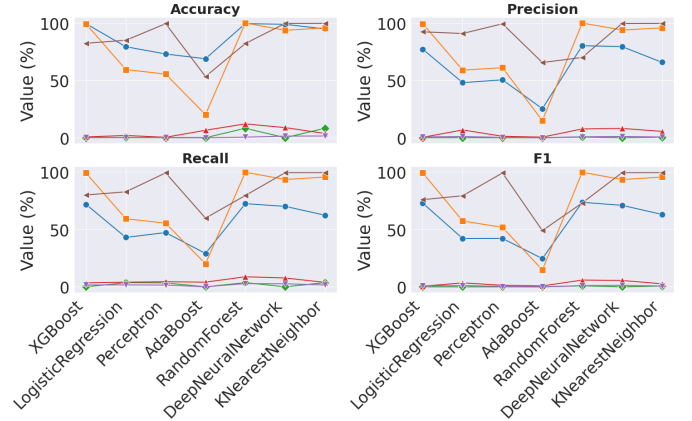
Classical Sampler	Fitting Duration (Seconds)	Label Classes	GAN Models	GAN Fitting (Seconds)	Generation Time (Seconds)
Random Undersampling	0.289	Binary	WGANGP	1463.43	0.033
	0.289	General		2792.73	0.035
	0.389	Specific		4194.21	0.036
Random Oversampling	1.424	Binary	CGAN	5449.47	0.347
	3.737	General		6262.72	0.116
	4.009	Specific		6473.58	0.030
SMOTE	352.28	Binary	CTGAN	5053.61	0.066
	2038.35	General		6019.70	0.070
	2128.92	Specific		7980.06	0.746
Cluster Centroids	5053.61	Binary	CWGANGP	3297.04	0.209
	48.70	General		3305.17	0.209
	5.80	Specific		9082.65	0.06

Oversampling stand out for their substantial improvements in accuracy, precision, recall, and F1 score. Although SMOTE incurs higher fitting durations, its effectiveness in improving model performance makes it a valuable method, especially for datasets with significant imbalances. Random Oversampling offers a good balance between time efficiency and performance gains. In contrast, Random Undersampling, while quick, provides only moderate improvements, and Cluster Centroids, despite its potential for significant enhancements, shows highly variable fitting times, which may limit its practical application. Overall, the choice of sampling method should consider both the performance improvements and the time required for fitting. For scenarios demanding high accuracy and robust performance, SMOTE and Random Oversampling are recommended, with the understanding that SMOTE requires a greater time investment. For quicker applications, Random Undersampling can be considered, albeit with moderate performance gains.

E. Autonomous IDS Results on GAN-based Balancing Models


Figure 8: IDS Performance on Binary Class (binary attack vs. benign traffic (1+1)) for *Synthetic* Balanced Datasets

Figures 8, 9 and 10 offer a comprehensive view of the performance metrics and fitting durations of various GAN


Figure 9: IDS Performance on General Class (general attacks vs. benign traffic (7+1)) for *Synthetic* Balanced Datasets

Figure 10: IDS Performance on Specific Class (specific attacks vs. benign traffic (33+1)) for *Synthetic* Balanced Datasets

models (WGANGP, CGAN, CTGAN, CWGANGP) across three classification types: binary (1:1), general (7:1), and specific (33:1). Additionally, the timing metrics for GAN fitting and data generation (as shown in the right side of Table II) provide insights into the computational demands of these methods.

From the performance metrics graphs, it is evident that balancing the dataset using GAN models significantly improves the IDS models' performance across all metrics—accuracy,

precision, recall, and F1 score—when compared to the unbalanced dataset. As we can observe in Figure 8, models trained on datasets balanced with CTGAN and CWGANP exhibit substantial improvements in performance. These models show a marked increase in accuracy, precision, recall, and F1 score compared to the unbalanced dataset, which performs poorly, especially in recall and F1 scores. WGANP and CGAN also improve performance, but their effectiveness varies, indicating that while they enhance model performance, they do not reach the levels achieved by CTGAN and CWGANP.

In the general attacks vs. benign traffic (7:1) classification (see Figure 9), the trend remains consistent. CTGAN and CWGANP continue to show high performance across all metrics, demonstrating their capability in effectively handling imbalanced datasets and improving IDS model accuracy. The unbalanced dataset underperforms significantly, highlighting the importance of using GAN-based methods for data balancing. WGANP and CGAN also contribute positively but show varied results, suggesting that their performance is influenced by the dataset's characteristics. Considering specific attacks vs. benign traffic (33:1) classification, the benefits of using GAN models are particularly pronounced. Figure 10 shows that CTGAN and CWGANP provide considerable improvements in performance metrics, even though their effectiveness slightly decreases compared to binary and general classifications. The unbalanced dataset shows the lowest performance, emphasizing the challenge of detecting specific attacks without proper data balancing. WGANP and CGAN show moderate to low performance, indicating that while they help, they are less effective for highly specific classifications compared to CTGAN and CWGANP.

Overall Discussion: When comparing the effectiveness of classical sampling algorithms and GAN models, it becomes clear that both approaches significantly enhance IDS model performance compared to unbalanced datasets. Classical sampling methods like SMOTE and Random Oversampling provide immediate and substantial improvements in performance metrics, particularly in recall and F1 score. These methods are effective and time-efficient, making them practical choices for data balancing. However, GAN models, especially CTGAN and CWGANP, offer a powerful alternative by generating high-quality synthetic data that further enhances model performance, particularly for complex and highly imbalanced classifications. Despite their longer fitting times, the ability of GAN models to produce nuanced synthetic data makes them invaluable for scenarios requiring robust data augmentation. This comprehensive evaluation underscores the effectiveness of both classical sampling and GAN-based methods in improving IDS model accuracy and reliability, with GAN models offering a sophisticated solution for complex data balancing challenges. While no single GAN-classifier combination emerged as the best universally, CWGANP paired with Random Forest consistently showed strong performance. Potential threats to validity include dataset dependency and the influence of hyperparameter tuning, which could affect the generalizability of the results.

VII. CONCLUSION AND FUTURE WORKS

In conclusion, our study shows that GANs significantly enhance IDS performance by effectively addressing imbalanced datasets. GAN-augmented datasets outperform classical sampling methods, improving accuracy and reliability. Future research will focus on real-time data generation, adaptive learning in IDS, and exploring GANs' broader impact across various cybersecurity domains and network environments.

REFERENCES

- [1] I. T. R. Center, "2023 data breach report." <https://www.idtheftcenter.org/publication/2023-data-breach-report/>, 2024.
- [2] IBM, "Cost of a data breach report." <https://www.ibm.com/reports/data-breach>, 2023.
- [3] M.-L. Messai and H. Seba, "Iot network attack detection: Leveraging graph learning for enhanced security," ACM ARES, 2023.
- [4] Iouliaou, Philokypros, Vasilakis, *et al.*, "A signature-based intrusion detection system for the internet of things," *Information and Communication Technology Form*, 2018.
- [5] Hidayat, A. M. Z., and A. Arshad, "Machine learning-based intrusion detection system: An experimental comparison," *Journal of Computational and Cognitive Engineering*, 2022.
- [6] A. A. Alfrhan, R. Alhusain, and R. Khan, "SMOTE: Class Imbalance Problem In Intrusion Detection System." <https://ieeexplore.ieee.org/abstract/document/9213728>.
- [7] A. Esquivel *et al.*, "Enhancing network-edge connectivity and computation security in drone video analytics," in *2020 IEEE Applied Imagery Pattern Recognition Workshop (AIPR)*, 2020.
- [8] "GitHub - Towson-Research/GAN-Balancing-Datasets: COSC 490 Towson University — github.com." <https://github.com/Towson-Research/GAN-Balancing-Datasets>.
- [9] A. Esquivel Morel, Z. Murry, K. Kostage, C. Qu, and P. Callyam, "Enhancing drone video analytics security management using an aer paw testbed," *IEEE International Conference on Computer Communications*.
- [10] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment," *Sensors*, vol. 23, no. 13, 2023.
- [11] A. Al-Fuqaha, M. Guizani, *et al.*, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications." <https://ieeexplore.ieee.org/abstract/document/7123563/>.
- [12] P. Freitas *et al.*, "Unsupervised gan-based intrusion detection system using temporal convolutional networks and self-attention," *IEEE Transactions on Network and Service Management*, vol. PP, pp. 1–1, 12 2023.
- [13] W.-C. Lin, C.-F. Tsai, Y.-H. Hu, and J.-S. Jhang, "Clustering-based undersampling in class-imbalanced data," *Information Sciences*, vol. 409–410, pp. 17–26, 2017.
- [14] M. H. Shahriar, N. I. Haque, M. A. Rahman, and M. A. J. au2, "G-ids: Generative adversarial networks assisted intrusion detection system," 2020.
- [15] Y. Yu *et al.*, "Cwgan: Conditional wasserstein generative adversarial nets for fault data generation," in *2019 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, pp. 2713–2718, 2019.
- [16] Y. Bengio, "Deep Learning of Representations for Unsupervised and Transfer Learning — proceedings.mlr.press." <https://proceedings.mlr.press/v27/bengio12a>.
- [17] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," *CoRR*, vol. abs/1907.00503, 2019.
- [18] I. Gulrajani *et al.*, "Improved training of wasserstein gans," *CoRR*, vol. abs/1704.00028, 2017.
- [19] E. Buedi, A. Ghorbani, S. Dadkhah, and R. Ferreira, "Enhancing ev charging station security using a multi-dimensional dataset : Cicevse2024," 03 2024.
- [20] E. C. P. Neto *et al.*, "Ciciov2024: Advancing realistic ids approaches against dos and spoofing attack in iov can bus," *Internet of Things*, vol. 26, p. 101209, 2024.
- [21] Y. AI, "ydata-synthetic project: Synthetic data generators for tabular and time-series data." <https://github.com/ydataai/ydata-synthetic/tree/dev>.
- [22] SK-Learn, "imbalanced-learn documentation 2014; Version 0.12.2 — imbalanced-learn.org." <https://imbalanced-learn.org/stable/index.html>.
- [23] M. Grandini, E. Bagli, and G. Visani, "Metrics for multi-class classification: an overview," 2020.